Inteligencia Artificial y Neurociencias

Licenciatura en Tecnología Digital, UTDT, 2S-2026

Profesores: Agustín Gravano, Andrés Rieznik
Auxiliares: Gerardo Del Toro, Antonella Giordano Furchi

Fundamentos de LLMs #3

Arquitecturas de Transformers

Decoder (ej: GPT, Claude, Llama)

A partir de una secuencia de tokens (un prompt), genera un token de salida por vez, de manera iterativa y autorregresiva. Son modelos generativos y causales: la información fluye de izquierda a derecha.

Encoder (ej: BERT, RoBERTa, RoBERTuito)

Recibe como entrada una secuencia de tokens y los mapea a una secuencia de embeddings contextuales de la misma longitud, que contienen información de toda la entrada. No genera texto: se usa para clasificar o tomar decisiones sobre la entrada (p. ej., análisis de sentimiento, NER).

Encoder-decoder (ej: BART, Whisper)

Un encoder procesa toda la secuencia de entrada; un decoder genera la secuencia de salida un token por vez, condicionada por el encoder mediante cross-attention.

Ajuste fino supervisado (SFT)

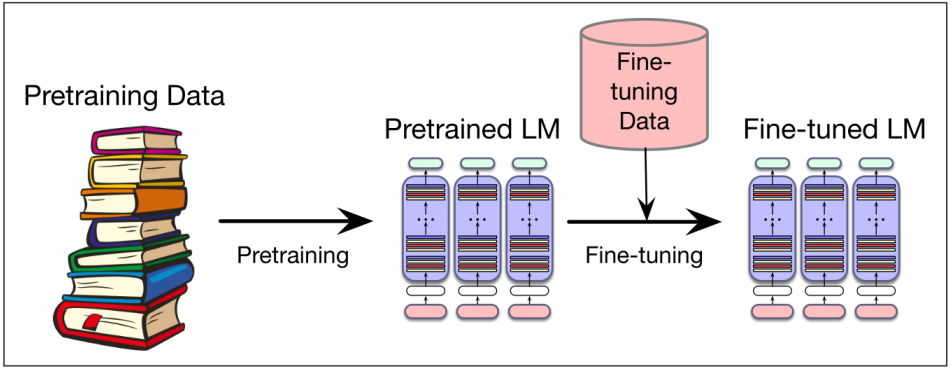


Figure 8.6 Pretraining and fine-tuning. A pre-trained model can be fine-tuned to a particular domain or dataset. There are many different ways to fine-tune, depending on exactly which parameters are updated from the fine-tuning data: all the parameters, some of the parameters, or only the parameters of specific extra circuitry.

LoRA (Low-Rank Adaptation)

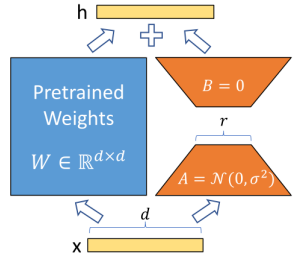
Fine-tuning completo: cada matriz $W \in R^{d \times d}$ requiere aprender una actualización ΔW del mismo tamaño.

Fine-tuning con LoRA (Edward Hu et al., 2021): se congela W y se aprende solo una corrección de rango bajo de la matriz, en lugar de actualizar todos los pesos.

$$W' = W + \Delta W = W + AB, \quad \text{con } A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}, r \ll d$$

Intuición: la actualización necesaria para adaptar un modelo a una tarea tiene rango intrínsecamente bajo. No hace falta mover los pesos en todas las direcciones, solo en unas pocas.

En el entrenamiento, solo A y B reciben gradientes; W queda congelada.



LoRA (Low-Rank Adaptation)

Ahorro (ej. GPT-3, $d = k = 12288$, $r = 8$):

- Full: $12288^2 \approx 151$M parámetros por matriz
- LoRA: $2 \cdot 12288 \cdot 8 \approx 197$K ($\sim 0.1\,\%$)
- Mayor eficiencia computacional: optimización solo sobre A, B
- Sin costo extra en inferencia: se fusiona $W' = W + AB$
- Adaptadores chicos e intercambiables (por tarea, dominio o dialecto)
- Rendimiento comparable al fine-tuning completo

QLoRA (Dettmers et al. 2023):

- Modelo base cuantizado a 4 bits, adaptadores LoRA entrenados en 16 bits
- Ahorra $\sim$4x de memoria (a costa de un entrenamiento un poco más lento)
- Permite fine-tunear modelos de hasta 65B en una sola GPU

Decoder-only Transformers

(LLMs generativos, causales, autorregresivos, left-to-right, GPT, etc.)

Scaling laws: Kaplan et al. (2020); Hoffmann et al. (2022)

¿Cuán grande debe ser un LLM? ¿Cómo elegir L, d, A, etc.? ¿Cuántos datos son necesarios?

La arquitectura (profundidad vs. ancho) importa poco en comparación con la escala.

Para una capacidad de cómputo (C flops), el tamaño del modelo (N parámetros) y la cantidad de datos (D tokens) deben crecer a la par.

La pérdida baja de forma predecible con el aumento de N y D, según esta fórmula:

$$\mathcal{L}(N, D) = \underbrace{E}_{\substack{\text{pérdida} \\ \text{irreducible}}} + \underbrace{\frac{A}{N^{\alpha}}}_{\substack{\text{pérdida} \\ \text{por modelo}}} + \underbrace{\frac{B}{D^{\beta}}}_{\substack{\text{pérdida} \\ \text{por datos}}}$$

$E, A, B, \alpha, \beta \in \mathbb{R}$ son constantes ajustadas experimentalmente.

Hoffmann22: $\alpha, \beta \approx 0{,}3$: Duplicar N o D reduce el término $\sim$20 %. Con $\sim$10x lo reduce a la mitad.

Este resultado sirve de guía para decidir cómo repartir el presupuesto: si tengo una capacidad de cómputo C, ¿cuáles serán los valores óptimos de N y D?

Scaling laws: Kaplan et al. (2020); Hoffmann et al. (2022)

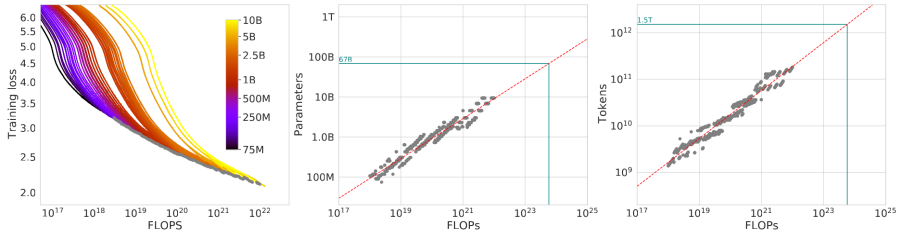


Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Tomado de Hoffmann et al. (2022)

Dado C fijo, si N y D están desbalanceados, pueden pasar dos cosas:

- N chico, D grande: el modelo no tiene capacidad para aprovechar tantos datos (subajuste)
- N grande, D chico: el modelo tiene capacidad de sobra y pocos datos (subentrenamiento)

In-context learning: Brown et al. (2020)

Few-shot prompting: Le pedimos al modelo que haga una tarea y le damos varios ejemplos para que entienda el patrón.

Ejemplo: Detección del idioma

```
Detectá el idioma de cada oración.

Oración: "the cat is on the table" → Inglés
Oración: "le chat est sur la table" → Francés
Oración: "die Katze ist auf dem Tisch" → Alemán
Oración: "el gato está sobre la mesa" → Español
Oración: "o gato está em cima da mesa" → Portugués
```

(Se muestra en **verde** lo que responde el modelo.)

El modelo infiere la tarea del patrón del prompt. No se actualiza ningún peso: todo ocurre en el contexto.

In-context learning: Brown et al. (2020)

One-shot prompting: Lo mismo, pero con un solo ejemplo. A veces alcanza.

Ejemplo: POS tagging

```
Dada una oración, etiquetá cada palabra con su categoría gramatical.

Ejemplo:
Oración: "el gato duerme"
Tags: "el → DET, gato → NN, duerme → VB"

Oración: "la niña corre rápido"
Tags: "la → DET, niña → NN, corre → VB, rápido → ADV"
```

(Se muestra en **verde** lo que responde el modelo.)

In-context learning: Brown et al. (2020)

Zero-shot prompting: Solo describimos la tarea.

Ejemplo: Clasificación de sentimiento

```
Clasificá el sentimiento de la siguiente oración como Positivo, Negativo o Neutro.

Oración: "La película estuvo bastante aburrida."
Sentimiento: Negativo
```

(Se muestra en **verde** lo que responde el modelo.)

A partir de este trabajo, se popularizó el **prompt engineering**: saber escribir prompts como una interfaz para obtener buenos resultados de los LLMs.

Instruction tuning: Wei et al. (2022)

Problema: Un LLM preentrenado no sigue instrucciones. Ante el prompt *"Traducir 'hola mundo' al inglés"*, el modelo continúa la oración en lugar de traducir. (Few-shot prompting alivia el problema, pero no lo resuelve.)

Idea: Hacer fine-tuning supervisado con un dataset de pares (instrucción, respuesta):

Instrucción: *Traducir 'hola mundo' al inglés*
Respuesta: *hello world*

Mismo objetivo que en el preentrenamiento: pérdida de entropía cruzada, solo sobre los tokens de la respuesta:

$$\mathcal{L}_{CE} = -\sum_{t \in \text{respuesta}} \log P_\theta\left(y_t \,\middle|\, \text{instrucción}, y_{<t}\right)$$

Así, se busca que el modelo aprenda a responder a instrucciones. Pero, ¿de dónde sacamos suficientes datos para esto?

Instruction tuning: Wei et al. (2022)

Fuentes de datos:

- Instrucciones y respuestas escritas por humanos
- Datasets para tareas de NLP ya existentes, reformulados:
  - Análisis de sentimiento (ej: IMDB):

    Instrucción: *¿Es positiva o negativa la siguiente reseña? 'La película estuvo bastante aburrida.'*

    Respuesta: *Negativa*

  - NER (ej: CoNLL-2003):

    Instrucción: *Listá las personas, lugares y organizaciones del texto: 'Messi jugó en el Barcelona.'*

    Respuesta: *Personas: Messi. Organizaciones: Barcelona.*

  - Resúmenes (ej: XSum):

    Instrucción: *Resumí en una oración el siguiente artículo: <artículo>*

    Respuesta: *<resumen original del dataset>*
- Datos sintéticos generados por un LLM con few-shot prompting y bootstrapping iterativo desde unas pocas semillas humanas (Wang et al. 2023, Taori et al. 2023)

Alineación con RLHF: InstructGPT y ChatGPT (2022)

Reinforcement **L**earning from **H**uman **F**eedback

Busca alinear el comportamiento de un LLM a las preferencias y expectativas humanas.

Problema de Aprendizaje por Refuerzo:

- **Agente**: LLM causal (GPT)
- **Acción**: Elección del token a generar
- **Estado**: Contexto en el paso actual de decoding (o sea: historia de tokens)

El LLM recibe feedback de un modelo de recompensas y así va mejorando sus respuestas.

- InstructGPT (OpenAI, 03/2022): Ouyang et al. 2022
- ChatGPT (OpenAI, 11/2022): construido usando la misma metodología



Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*

Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens

Amanda Askell† Peter Welinder Paul Christiano*†

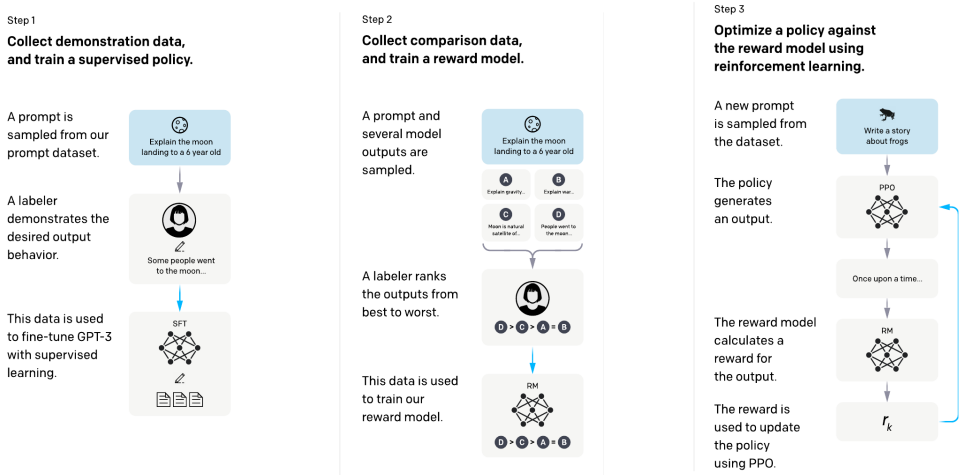
Jan Leike* Ryan Lowe*

OpenAI

Abstract

Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for aligning language models with user intent on a wide range of tasks by fine-tuning with human feedback. Starting with a set of labeler-written prompts and prompts submitted through the OpenAI API, we collect a dataset of labeler demonstrations of the desired model behavior, which we use to fine-tune GPT-3 using supervised learning. We then collect a dataset of rankings of model outputs, which we use to further fine-tune this supervised model using reinforcement learning from human feedback. We call the resulting models *InstructGPT*. In human evaluations on our prompt distribution, outputs from the 1.3B parameter InstructGPT model are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters. Moreover, InstructGPT models show improvements in truthfulness and reductions in toxic output generation while having minimal performance regressions on public NLP datasets. Even though InstructGPT still makes simple mistakes, our results show that fine-tuning with human feedback is a promising direction for aligning language models with human intent.

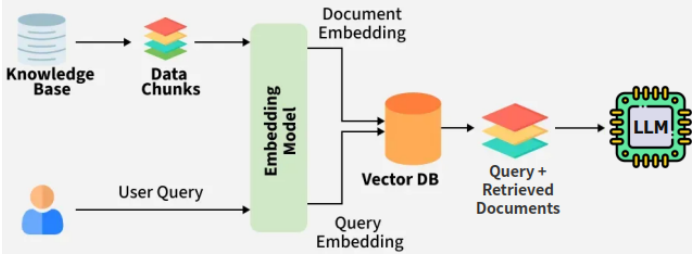
Alineación con RLHF: InstructGPT y ChatGPT (2022)



Obs: (1) es instruction tuning. En (2), los rankings se usan para asignar un puntaje a cada respuesta con el sistema Elo (ajedrez, FIFA). Con eso, el reward model aprende a asignar puntajes a respuestas. En (3), PPO es un método de gradiente de políticas del tipo actor-critic. Fuente: arXiv:2203.02155

Retrieval-Augmented Generation (RAG): Lewis et al. (2020)

Objetivo: Que un LLM responda sobre documentos externos (manuales, contratos, etc.)



1. Indexación (offline)
   a) Cargar los documentos y dividirlos en fragmentos
   b) Computar el embedding de cada fragmento (p.ej. con BERT)
   c) Guardar los pares (embedding, fragmento) en una *BD vectorial*

2. Recuperación y generación (online)
   a) Computar el embedding de la consulta del usuario
   b) Buscar los fragmentos más cercanos y copiarlos en el prompt
   c) Pedirle al LLM que responda la consulta usando esa info

Chain of thought (CoT)

Idea inicial (Wei et al. 2022): Pedirle al modelo (con few-shot prompting) que genere pasos intermedios de razonamiento para responder a una consulta.

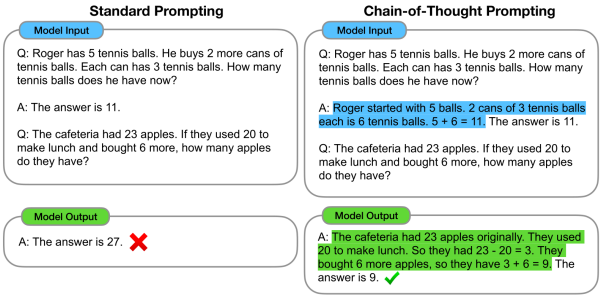


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

- Mejoras en matemática, sentido común y razonamiento simbólico.
- Intuición: Descompone un problema en pasos sencillos, cada uno más probable de salir bien.
- Pocos meses después, Kojima et al. (2022) mostraron que alcanza con agregar la frase *"Let's think step by step"*, sin ejemplos (zero-shot CoT).

CoT entrenado: OpenAI o1 (2024), DeepSeek-R1 (2025). Usando RL, el modelo aprende a descomponer problemas en cadenas largas: RLVR (Reinforcement Learning with Verifiable Rewards). Esto aumenta mucho el cómputo y la latencia en inferencia.

Algunas preguntas para las Neurociencias

1. En entrenamiento, un LLM ve billones (en español, 10^{12}) de palabras; un niño/a, apenas decenas de millones al llegar a los 10 años. ¿Cómo se explica semejante diferencia de eficiencia? ¿Qué le falta al modelo para acercarse al cerebro humano?

   Desde 2023, el BabyLM Challenge propone entrenar LLMs con cantidades de datos comparables a las que recibe un niño/a y estudiar qué aspectos del lenguaje emergen.

2. Para generar texto, un LLM elige, uno a uno, de entre los siguientes tokens más probables. Al producir lenguaje, ¿los humanos también elegimos las palabras de un modo simlar? ¿Tenemos algo parecido a la *temperatura* de los LLMs?

3. Los LLMs no incluyen módulos específicos para sintaxis, pero parecen aprender bien la estructura jerárquica del lenguaje. ¿Qué implica esto para el debate entre gramática universal y aprendizaje estadístico?

4. Atención en transformers y en Neurociencias: ¿comparten algo más que el nombre?

Algunas preguntas para las Neurociencias

5. Un LLM tiene dos formas de usar información nueva sin reentrenarse: in-context learning y RAG. En el cerebro, la memoria de trabajo y la memoria episódica del hipocampo cumplen funciones que recuerdan a estas dos. ¿Qué tienen en común y en qué difieren? ¿Qué le falta a los LLM para tener algo parecido a la consolidación?

6. ¿Se puede hacer algún paralelismo entre fine-tuning por un lado y plasticidad sináptica y olvido catastrófico por el otro?

7. ¿Las ideas de chain-of-thought se parecen al habla interna y/o al pensamiento deliberado?

8. Con los LLMs podemos hacer cosas que con el cerebro no: acceder a cada unidad, intervenir, ablacionar. ¿Qué experimentos de neurociencia del lenguaje (lesiones, estimulación, afasias) valdría la pena replicar en modelos? ¿Qué podríamos aprender?